



Master's in Data Science

Big Data Management

Cymatics

A Sound-Driven Pattern

Generation and Analysis Pipeline

Second Deliverable – Final Report

Arman Bazarchi · Brisa Fernanda Cisneros Cervantes

Contents

1	Instructions	1
1.1	Prerequisites and one-time setup	1
1.2	Applying MinIO access policies	1
1.3	Starting the stack	1
1.4	Running the pipeline	2
1.5	Interacting with the final data	3
2	Architecture Design	1
3	Trusted Zone	2
3.1	Design	2
3.2	Storage organisation	2
3.3	Pipeline and cleaning tasks	2
4	Exploitation Zone	3
4.1	Design	3
4.2	Generated features and embeddings	3
4.3	Storage organisation	5
4.4	Pipeline	5
5	Data Consumption	5
6	Data Governance	6
6.1	Data products	6
6.2	Implemented governance mechanisms	9

1 Instructions

In this section, we document how to install, start and interact with the pipeline, the project is fully self-contained and reproducible from a clean clone of the repository at BDM-Cymatics GitHub Repository. All services except the host-side Python orchestrator run inside Docker.

1.1 Prerequisites and one-time setup

The host machine needs Docker and Docker Compose, Python 3.10 or newer, the `ffmpeg` binary (required by the cold ingestion path) and a working microphone for the warm path, the hot path and the interactive consumption tasks. After cloning the repository, environment variables are configured by copying the provided example file and filling in the credentials:

```
cp env.example .env
pip install -r requirements.txt
```

The `.env` file declares the variables listed below:

Variable(s)	Purpose
MINIO_ENDPOINT, MINIO_ACCESS_KEY, MINIO_SECRET_KEY	MinIO root endpoint and credentials
MINIO_PIPELINE_ACCESS_KEY/SECRET_KEY	Pipeline role (RW ingestion & processing)
MINIO_ANALYST_ACCESS_KEY/SECRET_KEY	Analyst role (read-only consumption)
KAFKA_BOOTSTRAP_SERVERS	Kafka broker address
MILVUS_HOST, MILVUS_PORT	Milvus vector-DB connection
FREESOUND_API_KEY	Freesound API key (required for ingest)
AIRFLOW_USERNAME, AIRFLOW_PASSWORD	Airflow admin credentials
SONAR_TOKEN, SONAR_JDBC_*	SonarQube authentication

Table 1: Environment variables

1.2 Applying MinIO access policies

Before any ingestion can take place, the four MinIO IAM users (`pipeline_admin`, `data_engineer`, `data_scientist` and `analyst`) declared in `.env` must be created on the running MinIO instance, since the pipeline scripts authenticate as `pipeline_admin` and the consumption layer authenticates as `analyst`. Running this step is required once with:

```
python governance/data_security.py
# choose [1] Apply security policies
```

The script attaches the bucket-level access matrix described in data security section and uses `mc admin` inside the MinIO container, also creates a dedicated `governance-zone` bucket in MinIO if not available, It only needs to be repeated if the MinIO data plane is wiped (for example through `docker compose down -v` or with a manual deletion of the `./minio` bind mount).

1.3 Starting the stack

The entire infrastructure is brought up with a single command:

```
docker compose build
docker compose up -d
```

Service	Endpoint	Role
minio	:9000 / :9001	Object store backing every zone bucket; S3-compatible API.
zookeeper	:2181	Coordination service required by Kafka.
kafka	:9092	Event broker used by the hot-path ingestion stream.
kafka-ui	:8085	Web UI for inspecting Kafka topics and messages.
spark-master	:7077 / :8081	Spark cluster master and web UI.
spark-worker-1, spark-worker-2	—	Two Spark worker nodes for distributed batch processing.
spark-history	:18080	Spark history server for post-mortem inspection of jobs.
trusted-spark-batch	on demand	Containerised Spark batch for the trusted-zone cleaning stage.
exploitation-spark-batch	on demand	Containerised Spark batch for the exploitation-zone spectral features.
milvus	:19530	Vector database hosting the three embedding collections.
milvus-etcd	—	Metadata store required by Milvus.
milvus-minio	—	Dedicated MinIO instance backing the Milvus data plane.
attu	:3000	Milvus administration UI.
streamlit	:8501	Unified data-consumption and governance dashboard.
airflow-webserver	:8080	Airflow web UI for DAG management and monitoring.
airflow-scheduler	—	Airflow scheduler that triggers DAG executions.
airflow-init	one-shot	Initialises the Airflow metadata database on first run.
airflow-postgres	—	PostgreSQL database backing Airflow.
sonarqube	:9090	Code-quality server scanning the Python sources.
sonar-postgres	—	PostgreSQL database backing SonarQube.

Table 2: Containerised services declared in `docker-compose.yml`.

1.4 Running the pipeline

Recommended execution mechanisms is the provided interactive command-line orchestrator:

```
python orchestrate.py
```

Here we provide nine flows together with a live CPU / memory monitor. Every script can also be invoked directly from the command line.

#	Flow	Description
1	Warm path	Record a 5-second clip from the microphone, preview the cymatics, ask the user to approve or reject, and store the audio.
2	Hot path	Kafka producer (live cymatics preview, upload every 5 seconds) and consumer running in parallel.
3	Cold path – Freesound	Batch ingestion from the Freesound REST API; prompts for the batch size.
4	Cold path – ESC-50	Batch ingestion from the ESC-50 dataset; prompts for the batch size.
5	Trusted zone	Spark cleaning batch followed by Python cymatics PNG / MP4 generation and metadata persistence.
6	Exploitation zone	Spark spectral features, Python MFCC / harmonic features, Parquet-to-Delta sync, Milvus embedding ingestion and classifier-head training.
7	SonarQube analysis	Launches sonar-scanner against the repository and prints a code-quality report.
8	Data consumption	KPI discovery, audio similarity search, cymatics pattern search, metadata semantic search, all in Streamlit dashboard.
9	Data governance	Great Expectations data-quality checkpoints, MinIO role-based access control, cross-zone lineage tracking.

Table 3: Flows exposed by the command-line orchestrator (`orchestrate.py`).

The pipeline can also be orchestrated via Apache Airflow at <http://localhost:8080>:

`cold_freesound_ingestion` DAG is scheduled weekly, and `trusted_zone_processing` runs every two weeks (so that at least 2 batches of audios are ingested for processing), and `exploitation_zone_processing` runs every fifteen days (after trusted zone applies processing and records are ready for embedding). This way we guarantee that each stage runs only after a sufficient volume of data has been provided.

1.5 Interacting with the final data

Curated data can be consumed via the command-line orchestrator option 8, which exposes the four downstream tasks individually: a set of pre-defined KPIs for analysis, an audio similarity search that records from the microphone, a cymatics pattern search that accepts an image, or a microphone recording or a free-text description, and a natural-language metadata search. These tasks are also available in Streamlit dashboard, which can be opened directly at <http://localhost:8501> when the containerised service is running, or launched explicitly on the host with

```
streamlit run data_consumption/data_consumption_all.py
```

The dashboard includes one tab per task and an additional tab that surfaces the governance reports (data quality, lineage and access policies), it authenticates against MinIO with the `analyst` read-only credentials.

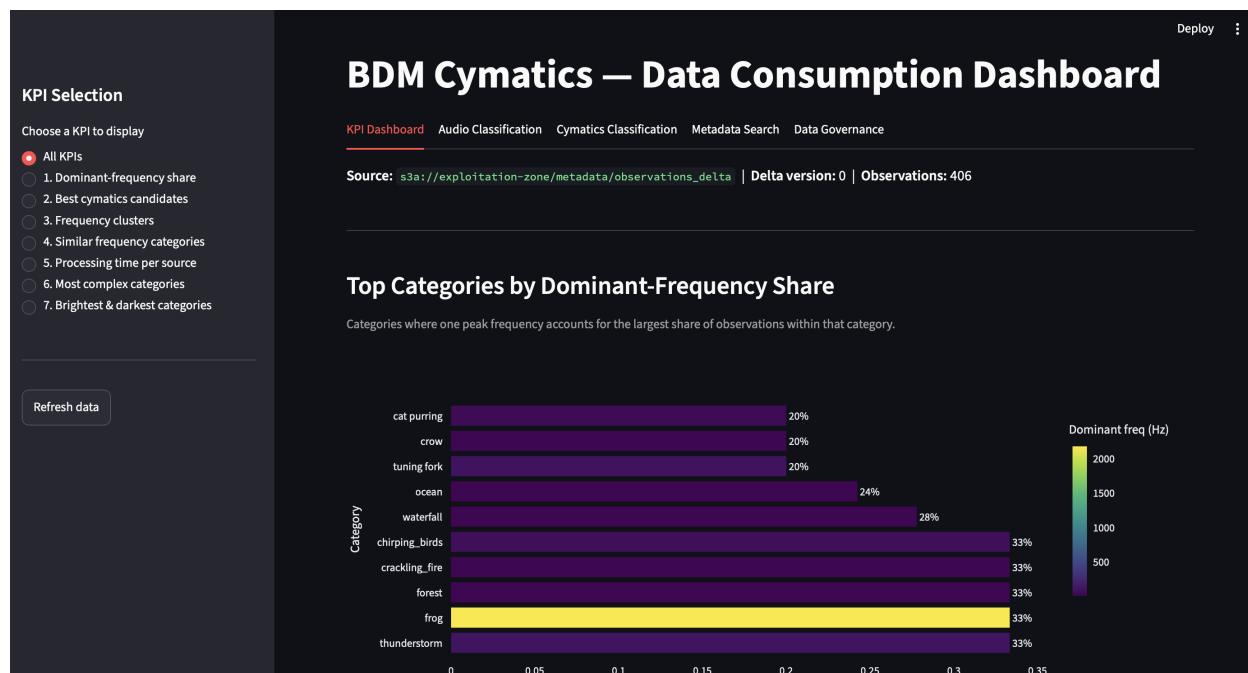


Figure 1: designed Streamlit dashboard

2 Architecture Design

We continue to follow the object-storage-centric approach described as Option 1 in the project statement and partially incorporate Option 2 by introducing a vector database for the exploitation zone. MinIO acts as the central object store across zones, Apache Spark performs distributed batch processing (to handle processing more efficiently) inside Docker, Delta Lake provides the analytical storage format for structured metadata at every stage (to handle queries efficiently), Milvus stores the embeddings (for efficient similarity searches), Kafka transports the hot-path event stream, and Airflow together with a custom Python orchestrator drive both scheduled and manual executions, via Streamlit we provide interactions with all data consumption tasks and data governance monitoring. Each zone has a separate bucket in MinIO that stores both unstructured assets and a tabular metadata table, as CSV, Parquet and a Delta Lake table. Embeddings, which are inherently high-dimensional and frequently queried by approximate-nearest-neighbour, live in a dedicated Milvus instance backed by its own MinIO (to keep them separately for their use cases), Code quality can be monitored by SonarQube, which surfaces bugs, vulnerabilities, code smells, duplication and a maintainability rating. The scanner is wired directly into the orchestrator so that a single command produces and displays a fresh report against the `bdm-cymatics` project.

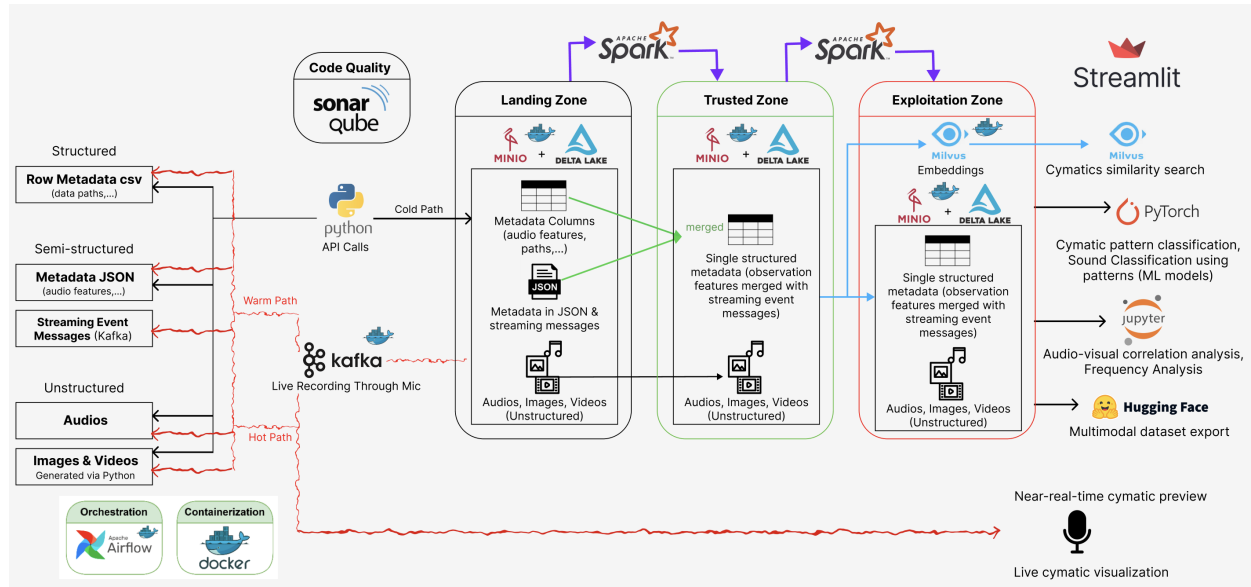


Figure 2: Architecture of the Cymatics pipeline.

Layer	Structured Storage	Vector Storage	Processing
Landing	CSV / Parquet / Delta	—	Python, Kafka
Trusted	CSV / Parquet / Delta	—	Spark, Python
Exploitation	CSV / Parquet / Delta	Milvus (3 collections)	Spark, Python, PyTorch
Consumption	KPIs / dashboard	Milvus ANN search	Pandas, Streamlit, PyTorch

Table 4: Summary of the implemented architectural layers and their associated technologies.

We separated processing stages by creating MinIO buckets for each zone. At the end of every processing run the structured metadata is synchronised to a Delta table so that analysts can query the latest data. Kafka events produced by the hot path are first archived inside the landing zone and later flattened and merged into the structured schema during trusted-zone processing, since the Kafka message schema is small and stable (timestamps and device identifiers), we prefer a tabular representation rather than a separate semi-structured store. Unstructured assets (audio recordings, cymatics images and videos) remain in MinIO under deterministic paths keyed by

peak frequency and UUID, which provides a natural grouping for downstream visual and acoustic analyses.

3 Trusted Zone

3.1 Design

Our trusted zone is responsible for transforming raw landing-zone observations into clean, schema-aligned representations. We follow the heuristic recommended in the project statement and restrict ourselves to transformations that are information-preserving and reversible, postponing every operation that would encode assumptions about a specific downstream task to the exploitation or consumption layers. This means deduplicating on the UUID primary key, casting numeric fields to their proper type, flattening the Kafka event payload into the tabular schema and removing rows whose underlying audio is missing or corrupted.

In addition to cleaning, in trusted zone we also generate the cymatics images and videos, each audio file is processed by the cymatics rendering engine (`shared/cymatics_engine.py`), which produces a 2048×2048 PNG image and an MP4 video, together with two cymatics quality metrics, namely the `symmetry_score` and the `pattern_stability_score`. This design choice avoids having to regenerate identical assets for every downstream task.

3.2 Storage organisation

The `trusted-zone` MinIO bucket follows a clean separation between unstructured assets and metadata. The directories `audio/`, `images/` and `videos/` contain the WAV, PNG and MP4 files grouped by dominant peak frequency and UUID, which preserves the locality of related observations. The `metadata/` directory hosts the trusted-zone observation table as CSV, Parquet and the Delta Lake table, together with a `pending_workset.json` consumed by the Spark batch that coordinates incremental work. The Delta table is updated at the end of every run with the latest curated version and is materialised using a typed schema declared in `shared/sync_delta.py`: floats for scores and durations, integers for byte sizes, and strings for identifiers and asset paths. This typing was added specifically to overcome the all-strings default produced by reading from CSV.

3.3 Pipeline and cleaning tasks

The trusted-zone pipeline is structured in two stages. The first stage is a distributed Spark batch that runs inside a dedicated container (`trusted-spark-batch`): it reads the landing-zone CSV through the S3A connector, deduplicates against the trusted-zone metadata using the UUID primary key, enforces the trusted schema, merges the Kafka event metadata for hot-path rows, and writes the resulting UUIDs to a pending workset on MinIO. The second stage continues on the host in Python: for every pending UUID it downloads the audio from the trusted bucket, invokes the cymatics engine to produce the corresponding image and video, computes the cymatics quality metrics, and appends the new records to the CSV, the Parquet and the Delta table. This way we have separated batch and per-row processing to run them independently. Execution can be triggered either manually through the orchestrator (`python orchestrate.py`, option 5) or automatically through the `trusted_zone_processing` Airflow DAG, which runs every two weeks so that at least two landing-zone batches that must be efficient for these processing, are ingested before each execution.

4 Exploitation Zone

4.1 Design

The exploitation zone curates the trusted data for analysis needs. We create a single wide table that joins every trusted-zone attribute with the audio features that we derive in this zone, this way we remove the need for joins at consumption time and having one central table for audio records fits more with our scope and data, also pre-joined tables reduce query complexity and latency. In addition to the tabular representation, the exploitation zone computes three families of embeddings (audio, text and cymatics image) and stores them in a Milvus vector database, so that downstream similarity-search and classification tasks can be served as cheap approximate-nearest-neighbour(ANN) look-ups.

4.2 Generated features and embeddings

The Spark batch implemented in `spark_exploitation_zone.py` extracts eight global spectral descriptors per recording.

Feature	Brief overview
spectral centroid	“Centre of mass” of the spectrum; higher values correspond to brighter sounds.
spectral bandwidth	Spread of the spectrum around the centroid;
spectral rolloff	describes where the bulk of the signal sits.
spectral flatness	values close to 1 indicate noise-like content, values close to 0 indicate tonal content.
signal energy	Sum of squared samples; coarse proxy of how loud and how long the recording is.
spectral entropy	high entropy means complex, noise-like sounds, low entropy means simple tonal sounds.
zero-crossing rate	Average number of sign changes per second.
loudness	Energy expressed in decibels (dBFS).

Table 5: Spark-derived spectral descriptors.

Two additional features, namely the MFCC coefficients (captures the timbral fingerprint of the recording) and the harmonic-energy ratio (indicating how tonal versus inharmonic the sound is), are computed in Python because they involve per-frame state that is not suitable for Spark. After the tabular features are produced and persisted, the same script triggers Milvus embedding generation, three Milvus collections share an HNSW index with cosine similarity ($M = 16$, `efConstruction = 256`) and use idempotent upserts (update on change) keyed on UUID, which allows to re-run the process without producing duplicates.

Collection	Model	Dim	Purpose
<code>sound_audio_embeddings</code>	PANNs CNN14	2048	Audio similarity search and classification
<code>sound_text_embeddings</code>	all-MiniLM-L6-v2	384	Natural-language metadata search
<code>sound_cymatics_embeddings</code>	CLIP ViT-B/32	512	Image and text-to-image cymatics search

Table 6: Milvus collections.

After the embeddings are persisted, as one of our provided data consumption tasks, the exploitation pipeline closes with a lightweight supervised step implemented in `classifier_training.py`: a logistic-regression head is fitted on top of the frozen PANNs and CLIP embeddings to predict the **category** of each observation. The training runs only when at least two categories with three or more samples are available, uses an 80/20 train/test split together with five-fold cross-validation, applies `class_weight = "balanced"` to handle the Freesound class imbalance correctly, and persists the fitted pipeline together with a JSON metrics report under `exploitation-zone/models/`.

Only the audio and cymatics modalities are trained on, the text embedding is intentionally excluded since audio and cymatics classifications only are our main scope.

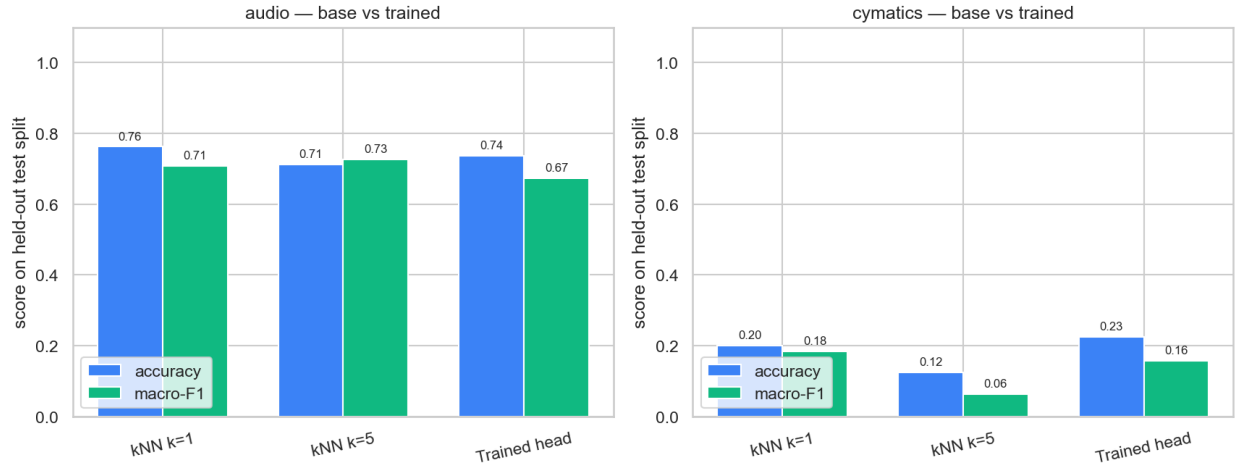


Figure 3: Base versus trained.

By training a small batch (400 records), and Comparing the trained classifier with two kNN retrieval baselines, we can see for audio, all three achieve similar performance, showing that the PANNs CNN14 embeddings already separate the classes effectively. For cymatics images, performance remains close to chance level for all methods, suggesting that CLIP does not capture cymatics patterns well and that larger training data or encoder fine-tuning may be required.

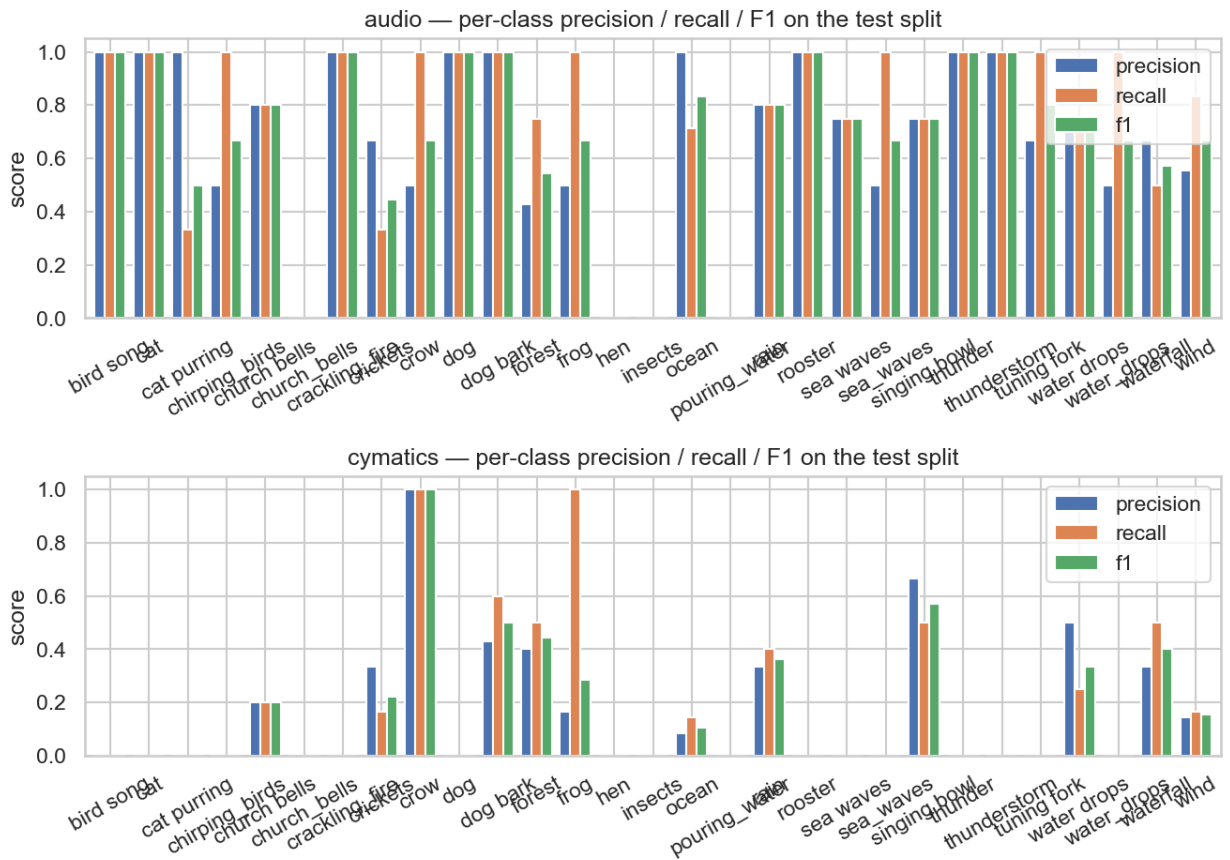


Figure 4: Per-class precision, recall and F1 on the held-out test split.

The audio model achieves consistently strong results across most classes, with some variation in underrepresented categories due to the small number of test samples. In contrast, the cymatics model performs poorly on many classes, suggesting a bigger training set or a more professional approach is required.

4.3 Storage organisation

Tabular data are stored in the `exploitation-zone` bucket under `metadata/` as CSV, Parquet, and a Delta Lake table. In the Delta sync we apply a per-zone schema cast, defined in, so that numeric columns land as `float64` or `int64` rather than as strings. The three Milvus collections live in a separate `milvus` service backed by its own MinIO storage (`milvus-minio`), keeping these resources separate avoids clutter, simplifies governance and access control, and allows the Milvus stack to be maintained or reset without affecting curated data. the Attu UI, reachable on port 3000, can be used for inspection and administrative checks.

4.4 Pipeline

The exploitation pipeline chains six steps: (1) execution of the Spark batch to derive spectral features, (2) per-row Python feature derivation (MFCC and harmonic-energy ratio), (3) appending the new rows to the CSV and Parquet, (4) synchronisation of the Parquet to the Delta table using the typed schema, and (5) construction or update of the three Milvus collections and (6) training and persistence of the classifier-head models for the audio and cymatics modalities. The script can be triggered manually through orchestrator option 6 or automatically through the `exploitation_zone_processing` Airflow DAG every fifteen days, one day after the trusted-zone DAG.

5 Data Consumption

The consumption layer reads the exploitation-zone Delta table through analyst-only S3 credentials and queries the three Milvus collections through their ANN interface. We implemented four downstream tasks, each task is exposed both as a CLI flow inside the orchestrator and as a tab in our provided unified Streamlit dashboard (`data_consumption_all.py`), which provides a coherent UI on top of the same Python functions.

KPI discovery. As one of the downstream tasks, we pre-defined seven queries over the exploitation-zone Delta table, each query returns a Pandas frame that the dashboard renders as a chart and offers as a CSV download. Together the queries cover the tabular analytical needs identified during development of this project.

ID	Query	Brief overview
KPI 1	Top categories by frequency share	Categories ranked by how dominant their most-repeated peak frequency is within the class.
KPI 2	Best cymatics candidates	Top recordings ranked by a combined cymatics quality score.
KPI 3	Top categories per frequency band	Within each low / mid / high frequency band.
KPI 4	Spectrally similar categories	Pairs of categories with the closest mean peak frequencies.
KPI 5	Processing time per ingestion source	Average trusted-zone processing duration grouped by ingestion paths.
KPI 6	Spectral complexity per category	Categories ranked by mean spectral entropy; high values mark noise-like sounds, low values mark tonal sounds.
KPI 7	Brightest vs. darkest categories	Categories with the highest and lowest mean spectral centroid, exposing tonal brightness or darkness.

Table 7: Predefined KPI queries.

Audio similarity search. In this task we allow recording five seconds audio from the microphone, then compute the 2048-dimensional PANNs CNN14 embedding, and feed it into the audio classification head that we trained in the exploitation zone, the embedding is then used to run an approximate-nearest-neighbour search against `sound_audio_embeddings` so that the top- k acoustically similar recordings are returned alongside the prediction, displaying to user both a labelled answer and the supporting evidence behind it. When the trained model is not yet available (for example, the exploitation zone has not been run on enough labelled data), the task falls back gracefully to the pure ANN-search behaviour.

Cymatics pattern search. For this task, we allow three search modes with the CLIP ViT-B/32 latent space: upload an image, or record audio (generates Cymatics image and embeds it), or type a natural-language description(text-to-image). For the image and audio modes the resulting 512-dimensional embedding is fed into the cymatics classification head trained in the exploitation zone, which returns the predicted cymatics category together with a confidence score. The same embedding is then used to run an approximate-nearest-neighbour search against stored image embeddings so that the top- k visually similar patterns are returned alongside the prediction. The text mode keeps its retrieval-only behaviour (no classifier is trained on it), returning the closest cymatics images via CLIP’s shared text-image space.

Metadata semantic search. We provided this task for text queries over the structured metadata, each of our observation is already summarized as a short natural-language description (category, source,...), embedded with all-MiniLM-L6-v2, and stored in Milvus. At query time the user’s text is embedded with the same model and matched by cosine similarity, this lightweight retrieval mechanism can later be consumed by a RAG-style chatbot for example.

The Streamlit dashboard is implemented as a containerized service in `docker-compose .yaml` that allows full interaction with implemented data consumption tasks and additionally provides a fifth tab that surfaces the governance reports, which we described in the next section.

6 Data Governance

6.1 Data products

Since purpose of our selected business domain, is to make audio observations and their visual cymatics representations searchable, classifiable and analyzable. Therefore in exploitation zone we ended up with five data products: the structured data product is for analytical uses, the three vector products are task-aligned for similarity search and retrieval, and a model registry product hosts the supervised heads trained on top of the audio and cymatics embeddings. Since each modality benefits from a specific embedding model and dimensionality, and is isolated, therefore governance layer can attach policies, owners and quality checks at the level of an individual data product.

sound_observations_delta(detailed example). This product is the analytical center of the domain. It stores a single observation row per UUID:

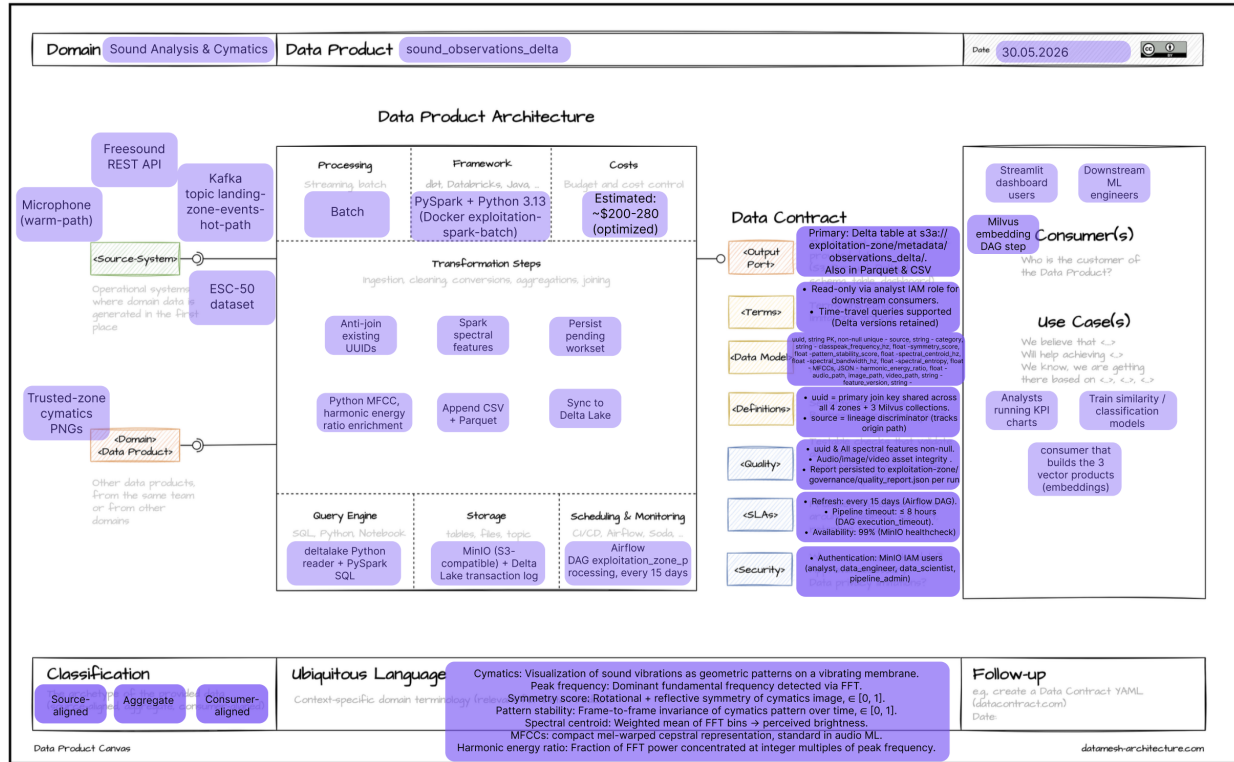


Figure 5: Data-product canvas for sound_observations_delta.

Column	Type	Group
uuid	string	Identifier (primary key).
category	string	Supervised label used by KPIs and classifier heads.
peak_frequency_hz	float64	Trusted-zone peak-frequency statistic.
peak_time_s	float64	Trusted-zone peak-frequency statistic.
peak_amplitude	float64	Trusted-zone peak-frequency statistic.
peak_rms	float64	Trusted-zone peak-frequency statistic.
all_peak_frequencies_hz	string	list of harmonics, kept as text.
symmetry_score	float64	Cymatics quality score in $[0, 1]$.
pattern_stability_score	float64	Cymatics quality score in $[0, 1]$.
spectral_centroid_hz	float64	Spark spectral feature.
spectral_bandwidth_hz	float64	Spark spectral feature.
spectral_rolloff_hz	float64	Spark spectral feature.
spectral_flatness	float64	Spark spectral feature.
signal_energy	float64	Spark spectral feature.
spectral_entropy	float64	Spark spectral feature.
zero_crossing_rate	float64	Spark spectral feature.
loudness	float64	Spark spectral feature.
MFCCs	string	JSON-encoded 13-dim vector (Python feature).
harmonic_energy_ratio	float64	Python feature.
audio_path	string	trusted-zone WAV location.
image_path	string	trusted-zone PNG location.
video_path	string	trusted-zone MP4 location.

Table 8: Columns of the sound_observations_delta product.

The product is owned by the `pipeline_admin` role and stored as a Delta Lake table on MinIO in exploitation zone. In data contract we guarantee a typed and fixed schema, which every fifteen days is updated with new records via exploitation-zone Airflow DAG. Quality is monitored by the Great Expectations (non-null UUID, no duplicates, scores in $[0, 1]$, valid asset paths). Primary consumers are the KPI module in Streamlit dashboard.

sound_audio_embeddings. Via this product we capture the acoustic signature of every recording. It is stored as a Milvus collection in which the primary key is the observation UUID and the vector field is a 2048-dimensional embedding produced by PANNs CNN14, from audio WAV records in trusted-zone, this model is pretrained on AudioSet, the largest weakly labelled environmental-audio records, which makes its embeddings directly relevant to the kinds of sounds we ingest (animals, water, ...). The 2048-dimensional output is rich enough to preserve fine acoustic distinctions and compact enough to keep the Milvus index and ANN look-ups cheap. Some columns are stored as keywords like the category, source, peak frequency, ... so that ANN searches can be filtered without scanning the table. The product is owned by the `data_scientist` role and data contract guarantees that vectors are L2-normalised, non-zero and of the exact declared dimensionality. Its primary consumers are the audio classification task (which retrieves the vectors to feed the trained logistic-regression head) and the audio similarity-search task (which uses the same vectors to return the top- k acoustically similar recordings).

sound_text_embeddings. This product makes our provided structured metadata searchable in natural language. For each observations, in exploitation layer we compose a short, human-readable description that summarizes the category, the ingestion source, the cymatics quality, ..., This description is embedded with all-MiniLM-L6-v2 to obtain a 384-dimensional vector, we selected this model because it provides a good balance between performance and efficiency. Its compact 384-dimensional embeddings allow fast similarity search and low memory usage in Milvus while still capturing the semantic relationships required for the retrieval task, larger models would increase inference costs without providing significant benefits for similarity-based retrieval. The product is owned by the `data_scientist` role and indexed with HNSW under cosine distance. In data contract we guarantee that descriptions are at most 4096 characters long and that vectors are L2-normalised. Its primary consumer is the metadata semantic search task, which provides a lightweight retrieval layer that can support a RAG-style chatbot in future.

sound_cymatics_embeddings. This product captures the visual signature of all observations. Each cymatics PNG generated during trusted-zone processing is embedded with CLIP ViT-B/32 to obtain a 512-dimensional vector, this model maps images and text into a shared embedding space, enabling both image-to-image and text-to-image queries, also remains efficient to run. The Milvus collection stores the vector together with the image path, so we can find and display the matching images with found results. The product is owned by the `data_scientist` role and indexed with HNSW under cosine distance, in this data contract as well we mirror the audio collection in terms of L2 normalization, idempotent upsert and refresh scheduling. Its primary consumer are the cymatics pattern-search task in its three modes (image upload, audio recording, and text to image search) and cymatics head training task.

classifier_models. This product captures the supervised heads we trained that map embeddings to category predictions. It is materialized as `StandardScaler` $\rightarrow$ `LogisticRegression` with balanced class weight, stored in MinIO exploitation zone bucket, both heads own JSON metrics reports that acts as the model card and holds details about the training like the set size and class distribution,... . We decided to train a thin linear head over a deeper from-scratch model because at our current scale a non-linear head would overfit. We did not include text data in this product because our main scope is audio and Cymatics classifications. The product is owned by the `data_scientist` role. In data contract we guarantee that each model carries a non-empty class list, a complete metrics report and a `model_version`, product is refreshed via the exploitation-zone DAG (every fifteen days, or on demand). Primary consumers are the audio-classification and cymatics-classification tasks.

6.2 Implemented governance mechanisms

We managed to implement four of the mechanisms suggested by the project statement, each is implemented as a script under `governance/` and is available through both orchestrator (option 9) and through the designed Streamlit dashboard.

Data quality with Great Expectations. In data quality aspects, we define three checkpoints aligned with the zone boundaries. The first checkpoint, between landing and trusted, validates following expectations:

- the `uuid` column is non-null on every row.
- there are no duplicated UUIDs within the metadata.
- the `peak_frequency_hz` value is strictly positive.
- the `source` field belongs to the allowed vocabulary (warm-path, hot-path, cold-Freesound, cold-ESC-50).
- the corresponding WAV file exists in the landing-zone MinIO bucket, with a valid RIFF/WAVE header and a non-zero file size.

The second checkpoint, between trusted and exploitation, ensures the presence of all trusted attributes, verifies the validity of the asset paths and checks that each cymatics image is a readable 2048×2048 PNG and that each video is a non-empty MP4. The third checkpoint inspects the Milvus collections to ensure that vectors have the declared dimensionality, are L2-normalised and are not all-zero. The benefit of this mechanism is that a single command tells us whether any zone has drifted from its contract.

Data security with MinIO IAM. In terms of security we define four organizational roles and attach a bucket-level access policy to each of them, and we use the `mc admin` CLI inside the MinIO container for attaching these policies, since the Python SDK only supports bucket-level policies and not user management. All provided pipeline scripts authenticate with the `pipeline_admin` credentials, while the Streamlit dashboard and the consumption tasks rely on the `analyst` credentials configured in `.env` for handling read/write access correctly. This is enforced at the application level using the dedicated read-only client. With this we ensure that each component operates with the appropriate permissions for its role., which prevents accidental writes from the consumption layer, the trade-off is the operational cost of maintaining two credential pairs in the environment. The resulting security audit report are stored in `governance-zone` bucket.

Role	Landing	Trusted	Exploitation	Governance
<code>pipeline_admin</code>	RW	RW	RW	RW
<code>data_engineer</code>	RW	RW	R	R
<code>data_scientist</code>	—	R	RW	R
<code>analyst</code>	R	R	R	R

Table 9: Role-based access matrix enforced by the MinIO IAM policies.

Lineage tracking. For this part we form, a UUID-indexed table that traces every observation across the three zones and the three Milvus collections. For each record we report which zones contain the observation, which assets were generated at each stage (WAV, PNG, MP4 and the three embedding types), the processing timestamps and a completeness flag indicating whether the full pipeline has finished. This allows us to detect ingestion gaps to trace cross-zones without any additional storage cost. Since the table is recomputed from existing metadata, full scans can become expensive once the dataset grows. As an example, the lineage entry below for a single `sea_waves` recording ingested via ESC-50 shows the full chain from raw audio to the three modality embeddings:

```

10. 062c6dbc-2b18-4cf9-88c1-...
Category:  sea_waves
Source:    ESC-50
Stages:    landing -> trusted -> exploitation
Progress:  [#####] 100%
Chain:
-> Ingested via ESC-50 -> landing-zone audio
-> Spark QA (dedup, schema) -> trusted-zone (image + video + peak detection, v2.0.0)
-> Spark spectral features + Python MFCCs -> exploitation-zone (v1.0.0)
-> PANNs CNN14 -> audio embedding (2048-dim)
-> all-MiniLM-L6-v2 -> text embedding (384-dim)
-> CLIP ViT-B/32 -> cymatics embedding (512-dim)

```

Data catalog (DCAT). Here to allow querying, validating, and consuming data catalogs by other systems, we decided to register our five data products in a machine-readable catalog, where canvas details for each product that we mentioned are recorded. Rather than deploying a heavy-weight catalog server such as Apache Atlas (which has no native connectors for MinIO, Delta Lake or Milvus, and would add several backing services to our already large stack), we decided to keep the catalog as an object in the governance bucket, being consistent with our object-storage-centric design. The catalog provides a readable JSON registry and a DCAT (Data Catalog Vocabulary) document to be easily accessed, where each product is a `dcat:Dataset`. The build step displays the live health and details of every product, then stores both files in governance zone. The trade-off is that the static registry must be updated when a new product is introduced. Like the other governance mechanisms, the catalog can be built and inspected either through the orchestrator (option 9) or through the Streamlit dashboard(recommended).

	Product	Type	Owner	Storage	Status	Detail
0	sound_observations_delta	structured	pipeline_admin	Delta Lake on MinIO	✓ available	Delta table present
1	sound_audio_embeddings	vector	data_scientist	Milvus (HNSW / COSINE)	✓ available	406 vectors
2	sound_text_embeddings	vector	data_scientist	Milvus (HNSW / COSINE)	✓ available	406 vectors
3	sound_cymatics_embeddings	vector	data_scientist	Milvus (HNSW / COSINE)	✓ available	406 vectors
4	classifier_models	model	data_scientist	MinIO object store	✓ available	audio + cymatics heads present

v sound_observations_delta — Sound observations (curated wide table)

Owner: pipeline_admin

Description: Single denormalised observation row per UUID: identifiers, peak-frequency statistics, cymatics quality scores, eight Spark spectral features and the two Python features (MFCC vector, harmonic-energy ratio), plus the audio / image / video asset paths. The analytical ground truth of the domain.

Storage: Delta Lake on MinIO (`exploitation-zone/metadata/observations_delta/` , Delta Lake (Parquet) + CSV mirror)

Schema: Typed schema enforced by `shared/sync_delta.py`

Data contract: Typed and fixed schema; refreshed every 15 days via the exploitation-zone Airflow DAG; quality monitored by Great Expectations (non-null UUID, no duplicates, scores in [0,1], valid asset paths).

Derived from: trusted-zone observations

Consumers: KPI discovery, Streamlit dashboard, lineage tracker, analysts

> sound_audio_embeddings — Audio embeddings (PANNs CNN14)

Figure 6: Streamlit-dashboard Data Catalog